



Contents lists available at ScienceDirect

Results in Engineering

journal homepage: www.sciencedirect.com/journal/results-in-engineering



Improving PCB defect detection using selective feature attention and pixel shuffle pyramid

Ka Chun Fung^{a,b,*}, Kai-Wen Xue^b, Cheung-Ming Lai^b, Kwan-Ho Lin^b, Kin-Man Lam^{a,b}

^a The Hong Kong Polytechnic University (PolyU), Hong Kong

^b Centre for Advances in Reliability and Safety Limited (CAiRS), Hong Kong

ARTICLE INFO

Keywords:

PCB defect detection
Convolution neural network
Multiscale feature fusion
Object detection

ABSTRACT

– Due to the ongoing miniaturization of electronic products and the use of miniature printed circuit boards (PCBs), existing AI-based defect detection methods have exhibited poor performance in detecting tiny PCB defects. This issue can potentially compromise safety, degrade manufacturing quality, and increase production costs. To tackle this problem, we propose two novel techniques for PCB defect detection, namely Selective Feature Attention (SF attention) and Pixel Shuffle Pyramid (PSPyramid). SF attention identifies important features from a pyramid feature map to fuse the semantic and spatial information, while PSPyramid effectively fuses semantic features to detect various types of defects on PCBs, especially tiny defects. Moreover, a customized training strategy, specifically for PCB defect detection, is devised. To evaluate the performance of our proposed algorithms, extensive experiments have been conducted on two well-known PCB datasets containing tiny defects: the DeepPCB and TDD datasets. Our proposed non-referential method achieves performance comparable to existing referential methods on the DeepPCB dataset, making it more feasible for industrial applications. Compared to state-of-the-art methods, our method reduces the error by 16%, in terms of AP50, on the TDD dataset. The experimental results demonstrate the effectiveness of our proposed method in improving the quality assurance process for PCBs in the electronics industry.

1. Introduction

The use of printed circuit boards (PCBs) has become ubiquitous in modern technology, as they serve as supporting structures and conductors for electronic components in a wide range of products, including mobile phones, computers, and home appliances. However, despite their importance, PCB production is facing various challenges, while defects on PCBs are the most pressing issue to be solved. Defects, such as short circuits, open circuits, and mouse bites, can be caused by a variety of factors, including human error and machine malfunction. These defects can significantly impact the proper operation of the final product, threaten the safety of users, and result in significant economic losses. In addition, the miniaturization of electronic devices has also made defects smaller and harder to detect.

To address this issue, various techniques have been developed to detect and prevent defects in PCB production. A popular solution is to replace manual quality inspection with automated machine inspection [1,2]. However, even with this approach, the problem of detecting defects still remains a significant challenge. Recently, the field of deep

learning has shown great promise in addressing this problem. For example, Domen et al. [3] proposed a convolutional neural network (CNN) to build a segmentation and decision module for identifying and classifying defects on PCBs. Similarly, Li et al. [4] proposed using YOLO-v4 [5] to detect silicon wafer cracks, whose appearance is uncertain and diverse. Other works, such as Ye [6], have used traditional methods, such as the Hough transform and watershed algorithm, in combination with CNNs to locate and classify defects. The difference between deep learning-based methods and traditional methods is that deep neural networks can extract more representative features, without relying on human engineering. Compared to handcrafted features, deep neural networks can be trained end-to-end, based on a loss function for feature learning and extraction. With proper design, deep models can achieve superior performance in almost all computer-vision tasks [7–10].

PCB defect detection has recently received significant research attention. To solve this challenging problem, different deep-learning methods have been proposed with different degrees of success [11]. For instance, Tang et al. [12] suggested using the VGG model for feature

* Corresponding author. The Hong Kong Polytechnic University (PolyU), Hong Kong.

E-mail address: ka-chun-ben.fung@connect.polyu.hk (K.C. Fung).

extraction and comparing the features of a test PCB image and a template PCB image. However, this method requires a template image, which limits its applicability in circumstances where templates are not accessible. Gao et al. [13] proposed using Faster R-CNN [14] for PCB defect detection, incorporating dilated convolutions for feature extraction with different receptive fields, and replacing Region of Interest (ROI) pooling with Gaussian ROI pooling to handle ROIs of different sizes. Saeed et al. [15] proposed using an autoencoder to restore PCB images and apply SSIM to detect defects.

Object detection models are very often used for defect detection. Therefore, the difficulties in PCB defect detection are mainly due to the fact that object detection methods are designed to address images in the natural domain and are not benchmarked on any PCB or industrial datasets, which makes the object detectors suboptimal by design. To make matters worse, the object detection models are evaluated using mean Average Precision (mAP), and the results look very promising and often reach above 95%. However, mAP is averaged by using different Intersection-over-Union (IoU) thresholds, which cannot perform well when the IoU threshold is set at a high value. Consequently, the detection models increase false negatives, i.e., resulting in a large number of missed defects, which is serious in quality control. Moreover, this also means a low defect recall. This problem seems trivial, but when it is translated into PCB quality control, it will be magnified into four problems.

1. The missed defects lead to decreased manufacturing quality.
2. Rework or scraping faulty products is necessary, so the production costs will increase.
3. Time to market is delayed as manufacturers need more time for quality control.
4. Safety is compromised, due to short circuits, overheating or component failure, and this poses potential health and safety risks to users.

The difficulty of PCB defect detection, based on existing deep neural networks, is that only spatial information (e.g., shape and location) is used to determine defects, but the pixel colour of a defect is highly similar to that of its neighbouring pixels. For instance, most object detection networks cannot differentiate pinholes from a circuit. In addition, most defects are tiny and close to each other, which is different from the target of modern object detection models. To address these problems, we propose a novel framework for PCB defect detection that exploits more spatial information than traditional methods. Our proposed model, which is based on Faster R-CNN [14], demonstrates improved performance on the DeepPCB and TDD PCB datasets, compared to other common and strong object detection models. The contributions of this paper are listed as follows.

1. Introduction of a novel aggregation method, namely Selective Feature attention with Pixel Shuffle Pyramid (SF-PSPyramid), which significantly improves the performance of detecting various types of PCB defects, particularly tiny defects with a high IoU threshold.
2. Selective Feature attention (SF attention) can reweight the importance of features, so important features can be highlighted.
3. PSPyramid is a learnable pyramid structure that can effectively extract and embed semantic information into a larger feature map.
4. A novel training method, which can efficiently integrate features from multiple scales, is devised.
5. Our proposed methods can achieve state-of-the-art performance on both the DeepPCB and TDD PCB datasets, demonstrating the effectiveness of the proposed method in the field of PCB defect detection and quality assurance.

2. Related works

In this section, we will first provide an overview of the object

detection networks used for defect detection. Then, existing feature aggregation methods and existing deep models for PCB defect detection are presented. Finally, techniques related to our proposed method, including pixel shuffle upsampling and attention mechanisms, are also presented.

2.1. PCB defect detection

Depending on whether a template image or a golden sample is required, PCB defect detection methods can be divided into two categories: referential methods and non-referential methods. In this section, we will provide an overview of these two categories of methods for defect detection.

2.1.1. Referential methods

Referential methods, as shown in Fig. 1, require a golden sample, in addition to the test samples, for the detection network. For better accuracy, the golden sample and test images are usually aligned before detection. However, in the production environment, it is rare to have a golden sample available for PCBs. Furthermore, it is not easy to align the golden sample with the test image because the product to be inspected is not always fixed at the same position on the production line.

Tang et al. [12] proposed a method for detecting PCB defects using a shared lightweight CNN to extract features from test and template images. The method utilizes a Group Pyramid Pooling (GPP) module to efficiently extract features with different receptive fields, allowing for the detection of defects of various scales. Template reference has also been used to tackle object detection with unseen texture and visual confusion [16]. The difference between a template image and a test image can also be used to diagnose the defect directly [17].

2.1.2. Non-referential methods

On the other hand, non-referential methods only require test images as input, making the detection more challenging as the network needs to locate and identify defects based on appearance, texture, and feature differences. These methods do not rely on a template or golden sample and are more applicable to real-world scenarios where perfect samples are difficult to obtain.

Ding et al. [18] proposed a method that utilizes k -means clustering to learn reasonable shapes for anchors, and then employs Faster R-CNN with the new anchors to detect defects. To address the issue of imbalanced class distribution, an online hard example mining (OHEM) scheme [19] is used to automatically select hard examples and improve detection results by retraining the detector with these examples. In addition to deep learning-based methods, Malge and Nadaf [20] proposed a method that uses mathematical morphology to perform segmentation on both the template and the test samples, followed by image subtraction to detect PCB defects. Zhou et al. [21] introduced a method that incorporates an attention module and an autoencoder, along with a dynamic thresholding technique, for segmenting bubbles in the TR-PCB imagery. Meanwhile, Yang et al. [22] presented a technique to reduce conflicting information from various levels, thereby accentuating the characteristics of PCB defects. Furthermore, Lim et al. [23] developed a multi-scale feature pyramid network aimed at improving the detection of small objects, complemented by an optimized IOU loss function to achieve more accurate defect identification.

2.2. Faster R-CNN

Faster R-CNN [14] is an improvement over its previous models, i.e. R-CNN and Fast R-CNN [14]. It is a mature and accurate detection method that has been widely used for object and defect detection [13,18,24,25], due to its accurate localization of defects. Faster R-CNN is considered a more accurate detector than previous one-stage models because a Region Proposal Network (RPN), which shares the full-image convolutional features with the detection network, is used to generate

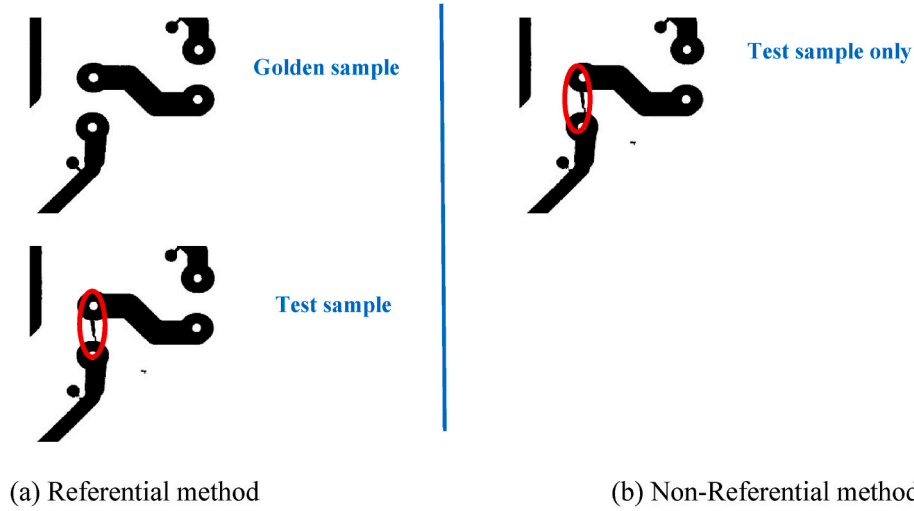


Fig. 1. Two approaches to PCB defect detection: (a) the referential method, and (b) the non-referential method. In the referential method, a golden sample is input with the test sample to help detect defects, which is not needed in the non-referential method.

region proposals. This enables almost cost-free region proposals, allowing it to learn features and localization from input images in an end-to-end manner. Therefore, Faster R-CNN is used as our baseline model, which is further improved for PCB defect detection.

Faster R-CNN [14] uses two fully connected layers after RPN to further classify the object class and regress the bounding-box parameters. The loss function used to train Faster R-CNN consists of two different loss functions: RPN loss (L_{rpn}) and Fast R-CNN Loss ($L_{fast\ rfcn}$). The RPN loss is based on a class-agnostic classifier, which classifies the foreground (i.e., defects) and background, and is used to output the bounding boxes for the foreground objects. The Fast R-CNN loss is used to further classify specific defect classes and refine the bounding boxes. In model training, the cross-entropy loss is used for the classification loss and smooth-L1 loss for the regression loss.

2.3. ResNet

ResNet is a commonly used backbone network for object detection because it is accurate, versatile, and can be easily fine-tuned on different datasets. The key innovation of ResNet is the residual connections, which help to learn and preserve significant features of the input images and solve the vanishing gradient problem. ResNet consists of a convolutional layer with batch normalization and ReLU, followed by Max Pooling and four ResNet blocks, i.e., C2, C3, C4, and C5. When the input passes through the blocks, the corresponding feature maps are gradually downsized. The final stage of the network contains an average-pooling layer and a fully connected layer. Finally, the output feature is elementwise added with the input feature. Because of its effectiveness and versatility, ResNet is used as the backbone of our designed model.

2.4. Feature aggregation

To achieve better performance in detecting objects of different sizes, it is important to aggregate features from a hierarchy of feature maps at different scales. Before feeding the aggregated or fused features to a fully connected layer for classification and detection, it is necessary to normalize the features to a fixed size, which can be done using Spatial Pyramid Pooling (SPP) [26], ROI pooling, or ROI align. The pyramid structure, as in FPN [27], NasFPN [28], and PAN [29] has high-resolution features being processed first, which are then downsampled repeatedly to form high-semantic representations of the input. Each level of the pyramid captures information at different scales, so objects of different sizes can be detected.

Feature Pyramid Network (FPN) [27] is frequently employed for object detection tasks. By building a pyramid of features from different layers of a deep neural network, the high-resolution but low-semantic features in lower layers are combined with low-resolution but high-semantic features present in the top layers of the network. This results in features from different layers containing high-semantic information. The benefit of FPN is its ability to use both high-level and low-level information together, which helps in detecting objects of different sizes. In this paper, we have used the feature aggregation process with an enhanced top-down path to add semantic information to the largest feature maps.

2.5. Pixel shuffle upsampling

To extract semantics from a feature map, pixel shuffling is used in our model to enhance the upsampling of semantic-rich feature maps. In PCB defect detection, the upsampling step in the FPN is very important, as it allows the propagation of semantic information. Therefore, a better upsampling method can make the FPN contain more effective information. In image processing and computer vision, one of the upsampling techniques is called pixel shuffle upsampling [30]. When used with CNNs, the architecture is especially effective for applications requiring image classification and recognition. To create high-resolution images from low-resolution inputs, pixel shuffle upsampling is often employed in the context of image super-resolution and restoration. The ability to efficiently upsample an image with better quality is a benefit of pixel shuffle upsampling. This is because images are upsampled by learned weights, rather than simple interpolation. On the contrary, bilinear or bicubic upsampling generates a higher-resolution image through pixel interpolation, and no new details are added.

2.6. Attention mechanisms

The attention mechanism in CNNs allows them to process data by focusing only on specific regions of the data. Instead of identically treating every part of the input, it enables the deep model to dynamically weigh each part differently. CNNs use channel attention and spatial attention as two primary forms of attention mechanisms.

One of the most commonly used channel attention mechanisms is based on the squeeze-and-excitation (SE) module [31]. In the squeezing step, the collection of statistics broken down by channel shows the distribution of activations across all channels. These channel-specific weights are then applied to the data in the excitation stage. As a

result, the model can selectively weigh the significance of various channels according to the patterns or features of the channels. Selective Kernel Networks (SKNets) [32] selectively weigh the significance of various kernels in the network to improve the effectiveness of CNNs. This enables the model to focus on the most crucial kernels and disregard extra kernels that do not contribute to the final prediction.

On the other hand, spatial attention focuses on specific spatial locations in the input. With the aid of this attention mechanism, a deep model can selectively weigh the significance of certain regions, depending on whether the defects are present in those regions. In the Convolutional Block Attention Model (CBAM) [33], channel attention is used first, followed by spatial attention, to gain the benefits of both types of attention. However, this means that information may be filtered out before being passed on to spatial attention. Our proposed attention mechanism can selectively weight the semantic-rich feature map and the high-resolution feature map.

3. Proposed model

3.1. Architecture

We propose a new framework for defect detection, which explicitly explores the spatial differences of feature maps at different scales. The structure of our proposed model is based on Faster R-CNN, as shown in Fig. 2. ResNet50 serves as our backbone network. During training, the input is downsampled to different scales from the convolutional layers. To detect defects of different scales, feature maps from different layers of

ResNet50 are fused by our proposed SF-PSPyramid, which will be described in Section 3.2. This fusion combines high-level features from low-resolution feature maps and low-level features from high-resolution feature maps. Then, ROI alignment is employed to normalize the dimension of the fused features [34]. For the model head, we utilize two fully connected layers. One output is designated for classification and the other for regression. Furthermore, multi-scale training is employed, so each resized image is used, and kernels are trained to adapt feature maps of different sizes. To remove overlapped detections, we apply Soft-Non-Maximum Suppression (Soft-NMS) [35] to remove duplicate or highly overlapped bounding boxes.

3.2. SF-PSPyramid

3.2.1. General structure

SF-PSPyramid is a crucial component in our proposed architecture. As shown in Fig. 3, the structure contains a bottom-up pathway, formed by C1–C5, and a top-down pathway, formed by P1–P6. In the bottom-up pathway, feature maps are extracted from the C2, C3, C4 and C5 layers of the Resnet50 backbone. C1 is not used due to its large memory footprint and low semantic information. When we move from one layer to the next layer in the bottom-up pathway, the feature map is down-sampled by a factor of 2, and the number of feature channels is increased by a factor of 2.

The novelty of SF-PSPyramid mainly comes from the design of the top-down path. The top layer P5 is generated by passing C5 into a 3×3 depthwise separable convolution (MConv) and is then directly sent to

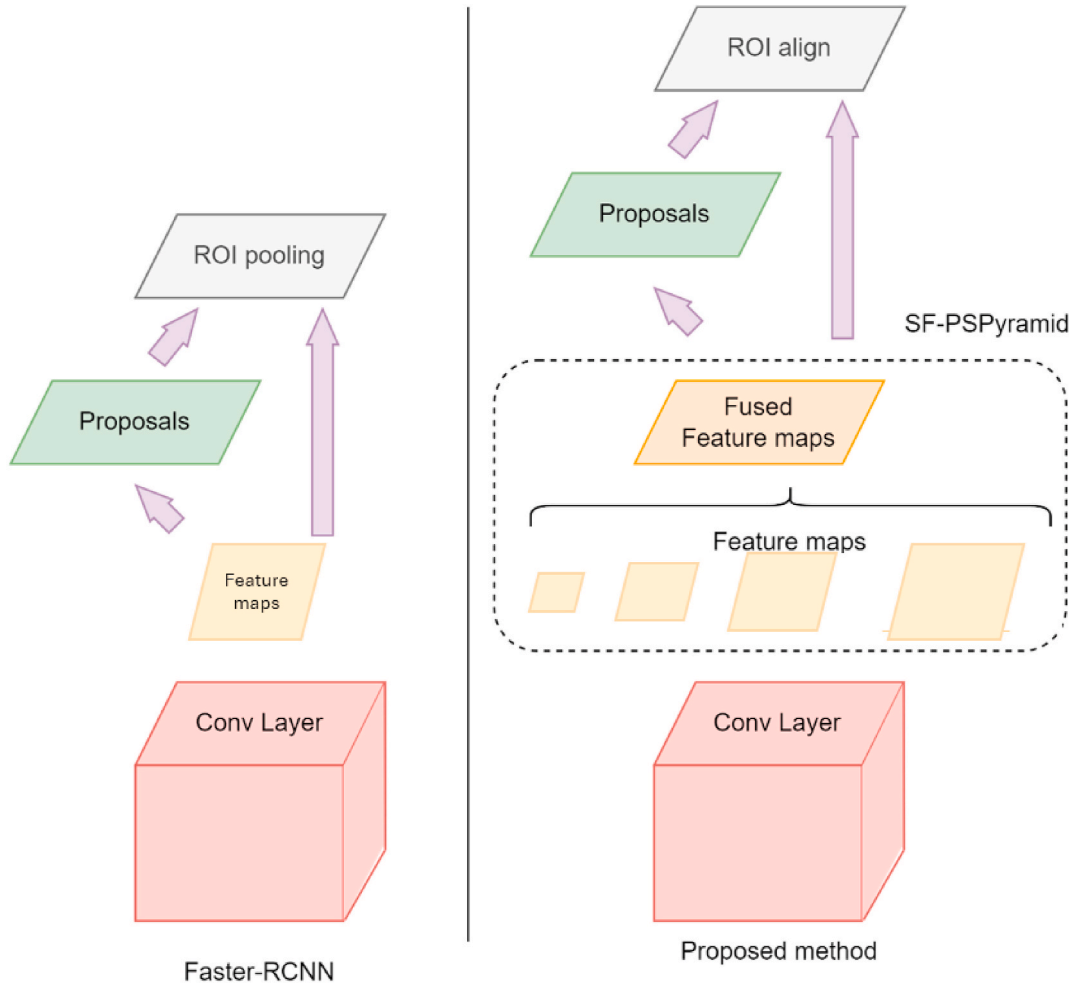


Fig. 2. Architecture of the proposed model. The proposed SF-PSPyramid in Faster R-CNN is shown in the dashed box on the right, which uses multiple feature maps in each layer to reconstruct the fused feature maps.

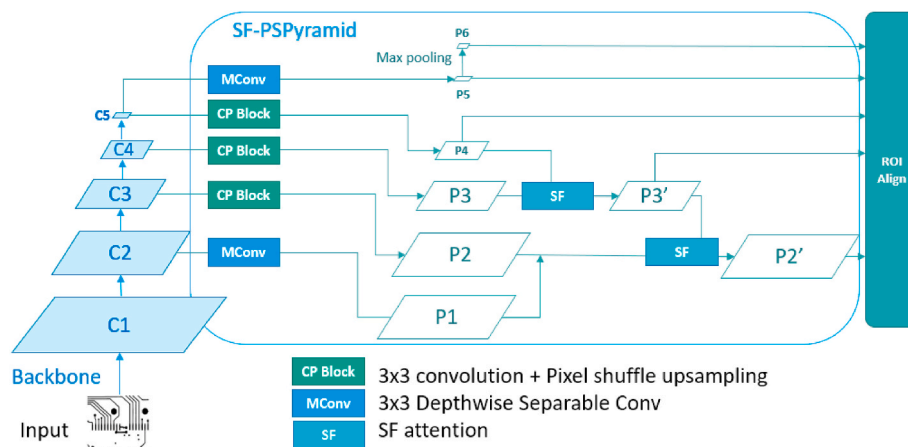


Fig. 3. Structure of SF-PSPyramid.

the ROI align layer because it has sufficient semantic information. P6 is created by passing P5 to a max pooling layer to downsample it. In the remaining top-down pathway, i.e., from low-resolution feature maps to high-resolution feature maps, the upsampled feature maps are denoted as P4, P3, and P2, which are generated from C5, C4, and C3, respectively, through a corresponding CP block. A CP block (Section 3.2.2), which is a learnable module, performs feature extraction and upsampling. Therefore, the deep feature map P4 inherits the high semantics from C5 and has a higher spatial resolution. Consequently, P4 is also directly passed to ROI alignment for defect detection. These three layers, i.e., P4, P5, and P6, contain high semantics, but have low resolution. They have limited ability to detect tiny defects.

Layers P2 and P3 only inherit the semantics from C3 and C4, respectively, which have insufficient semantics and need further refinement by injecting more semantic information from C5. Therefore, our proposed SF-PSPyramid adds higher semantic information to P2 and P3 to create feature maps P2' and P3', respectively, for defect detection. Specifically, P3' contains information from C4 and C5, while P2' contains information from C2, C3, C4, and C5.

The feature map P3' is generated by using SF attention to combine the feature map P3 with the higher-semantic feature map P4. SF attention (Section 3.3) combines a small feature map with a large feature map by selectively focusing on the semantics. Consequently, the semantics from a smaller feature map and the spatial content in a larger feature map can be fused in the right balance.

The lowest layer, $P2'$, is specifically designed to handle the problem of detecting tiny defects in PCB images. This layer is generated from three feature maps, including $C2$, $P2$ (extracted from $C3$), and $P3'$ (extracted from $C4$ and $C5$). In other words, $P2'$ contains high semantics with high spatial resolution. Detecting tiny objects is a challenge for modern object detection models, so our model focuses on the largest feature map, $C2$. This is because spatial information serves as important semantic information. To generate $P2'$, the output of $C2$ through $MConv$, i.e., $P1$, is first fused with $P2$ through addition. This fused feature has a high resolution but contains semantics from $C2$ and $C3$ only. To further improve its semantic content, this fused feature is combined with $P3'$, which contains the semantics from $C4$ and $C5$, through SF attention to generate $P2'$. Finally, $P6$, $P5$, $P4$, $P3'$, and $P2'$ are passed to the ROI align layer for defect detection.

The most significant characteristic of SF-PSPyramid is that most of the feature maps in the top-down pathway do not directly receive information from lateral convolutions. The P_i feature maps, $i = 2, 3, 4$, receive information mostly from the corresponding C_j feature map, $j = 3, 4, 5$. No P_i feature map has a direct connection to the C_i feature map. In this way, it forces the model to learn and use the semantics from deeper feature maps. The rationale for this design is that PCB defect images are not natural images and most of the targets, i.e., the defects, are tiny in

size. Therefore, focusing on shallow feature maps while using the semantics from deeper feature maps can assist in the detection of tiny PCB defects.

3.2.2. CP block

As shown in Fig. 4, a CP block consists of 3×3 convolutions and pixel shuffle upsampling. When pixel shuffle is applied to a feature map, the size of the feature map is doubled, while the number of channels is reduced to only a quarter of its original channel size (256). This process can more efficiently extract and retain the semantics in the feature map while performing upsampling.

The 3×3 convolution serves two functions in the architecture. First, it unifies all feature maps of different sizes in the pyramid to have the same number of channels of $4 \times c_{out}$ (1024). Hence, each layer can have the same number of channels and can be fused more easily. Second, the 3×3 convolutions can produce $4 \times$ more feature maps. The new feature maps are crucial for pixel shuffle upsampling because the corresponding points in the new feature maps are positioned as neighbouring points in the high-resolution feature map to be generated. Unlike linear interpolation, the upsampling process is learnable by using a convolutional kernel, so more details can be preserved or created.

3.3. SF attention

In the proposed SF attention, the upsampled feature map (Feature Map 1) and the current feature map (Feature Map 2), as shown in Fig. 5, are first added to create a fused feature map. Rather than directly using the fused feature map, global pooling is applied to create a representation with c channels. Then, this representation is further compressed into z channels to extract information from the channel dimension. After that, the representation is passed to another fully connected layer with softmax to decode the compressed vector back to the original number of channels. The expanded representation acts as the weights for the input feature map, and weighted summation is performed to obtain the final fused feature map.

The design of SF attention is based on a selective kernel network (SK Net) [32]. In SK Net, a feature map is split into two streams by using 3×3 convolutional kernels or 5×5 convolutional kernels. Then, the combined output is used to determine the weights for the weighted summation, so the feature maps can be combined more optimally. In our implementation, as shown in Fig. 5, the input contains two feature maps from two adjacent layers, where the feature map from the higher layer has a lower resolution. The higher layer is upsampled by SF to have the same resolution as the other input feature map. SF attention adaptively chooses weights from the two feature maps, based on their semantic and spatial information, and generates a fused feature map that optimally contains both types of information. SF attention uses two feature maps

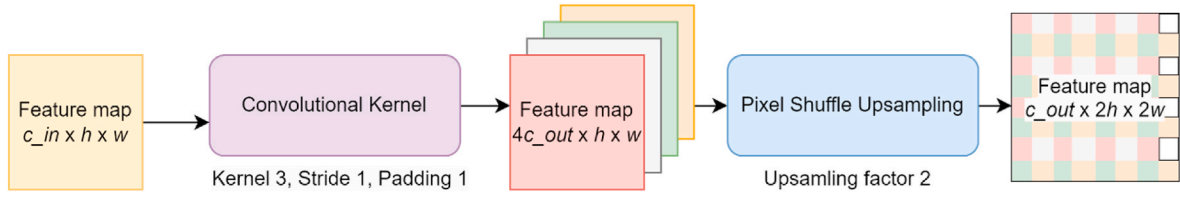


Fig. 4. The architecture of a CP block, where c_{in} and c_{out} denote the number of input channels and output channels, respectively. h and w denote the height and width, respectively, of the feature map.

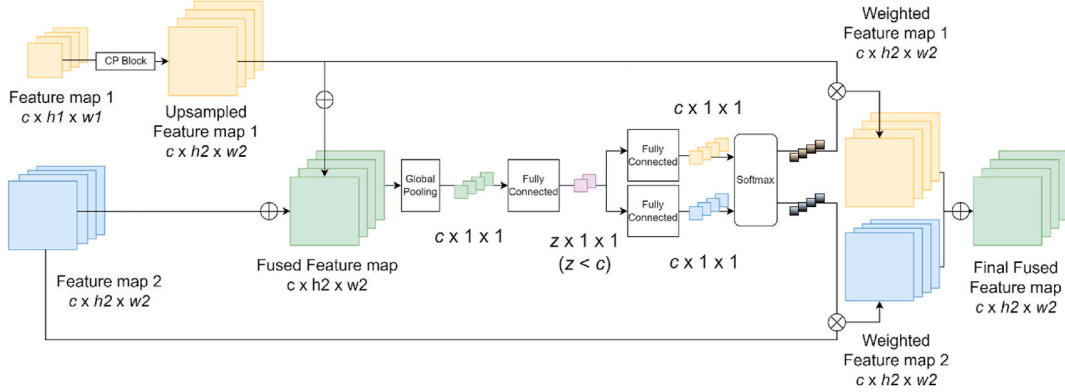


Fig. 5. Selective Feature attention (SF attention).

to generate an optimal fused feature map, so the attention mechanism can align the feature maps and select useful information from the two feature maps. An advantage of this proposed architecture is to create cross-scale attention inside the pyramid.

4. Experiments

4.1. Model settings

ResNet50 is the backbone used to extract image features at different levels for our defect detection model. To more accurately detect defects of different sizes, multiscale training is adopted, which is a commonly used training technique and works particularly well with our proposed SF-PSPyramid and Faster R-CNN. The scales used include 880×880 , 800×800 , 720×720 , 640×640 , 560×560 , and 480×480 . To remove overlapped detection results, Soft-NMS is used. The IoU threshold is set at 0.5. The learning rate is 0.02, the momentum is 0.9, the weight decay is 0.0001, and the optimizer used is Stochastic Gradient Descent (SGD). The step-based decay learning rate scheduler and the warmup scheme are used in training. The number of warmup iterations is 500 and the warm-up-ratio is 0.001. The number of training epochs is 12. For data augmentation, only random flip is applied, and no advanced data augmentation techniques are used. All experiments were conducted on a 40 Gb Nvidia A100 GPU.

When training a region proposal network, an anchor is defined as positive when its IoU with any ground-truth box is higher than 0.7. An anchor is negative if its IoUs with all ground-truth boxes are lower than 0.3. In order not to miss any defects, those anchors having the highest IoU with ground truths are positive when the IoU is higher than 0.3. In our extensive experiments, the cross-entropy loss is chosen as the classification loss and the L1 loss is chosen as the regression loss for estimating bounding boxes.

4.2. Datasets

The DeepPCB [12] and TDD PCB [18] datasets are used in our experiments. Some sample images of the two datasets are shown in Fig. 6.

In Ref. [12], a linear scan CCD camera was used to capture PCB images and a threshold was then set to binarize the images. The reason for binarizing the images is that PCB images have less diversity, in terms of shape, colour, and contour, so they can be represented as black-and-white images for defect detection. However, existing pre-trained object detection models are not designed for black-and-white images or PCB datasets. To expand the defect image dataset, synthetic defects are created, and data augmentation is used to expand the datasets. In addition, each PCB image with defects has a corresponding template image, and the PCB image and template image are strictly aligned. The training set contains 1000 images and the test set contains 500 images. The image size is 640×640 . DeepPCB contains the defects of “open”, “short”, “mouse bite”, “spur”, “pinhole”, and “spurious copper”. Tang’s method [12] used VGG-tiny to extract features from both the test image and the template image, followed by a group pyramid pooling (GPP) block. The pooling layers have different sizes, and the features are extracted based on different receptive fields to detect defects of different sizes.

In the TDD dataset, Ding et al. [18] extended the dataset by data augmentation, forming a dataset of 10,668 images. Each image is cropped into patches of size 600×600 . The TDD dataset contains the defects of “missing hole” (Fig. 6a), “mouse bite” (Fig. 6b), “open circuit” (Fig. 6c), “short” (Fig. 6d), “spur” (Fig. 6e), and “spurious copper” (Fig. 6f). In Ding’s method [18], Faster R-CNN was employed to find proper anchors through k -means clustering, and OHEM was applied to make the model pay more attention to hard examples. The challenge in this dataset is that the defects are all tiny. However, a weakness of using existing object detection models is that they were pretrained on large objects, such as cars and people.

4.3. Experiment results

4.3.1. Results of non-referential methods on the DeepPCB dataset

In addition to Tang’s method [12] and Ding’s method [18], we also compare our model with different implementations of one-stage detectors, including SSD [36], RetinaNet [37], and Cornernet [38], as well as two-stage detectors: FCCN-GWP [13] and Faster R-CNN [14].

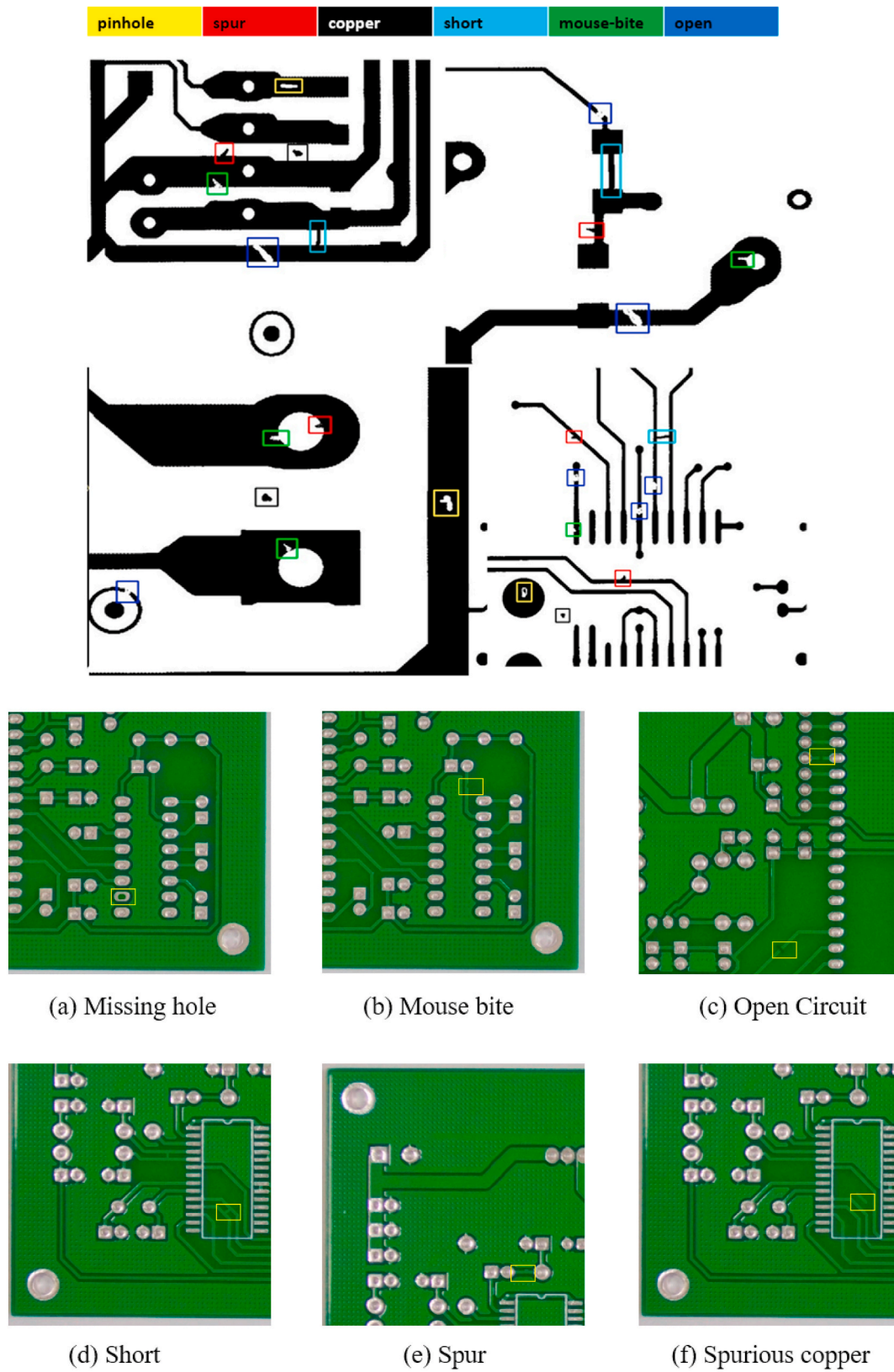


Fig. 6. Top: Samples from the DeepPCB dataset. Bottom: Samples from the TDD PCB dataset.

Following the previous state-of-the-art methods, Faster R-CNN in our proposed framework is also implemented using the backbone ResNet-50. Faster R-CNN, SSD, and Retinanet are also tested, all using the same ResNet-50 as the model backbone, while Cornernet used

Hourglass-104 as the model backbone. In FCCN-GWP, ResNet-50 was used as the backbone, and a feature collection and compression network was proposed to fuse the features generated by different receptive fields. Rather than using ROI pooling in Faster R-CNN, FCCN-GWP used

Gaussian weighted pooling to extract features from region proposals.

The performance of the different non-referential methods on the DeepPCB dataset is shown in Table 1. Faster R-CNN with our proposed SF-PSPyramid demonstrates superior performance over standard model architectures, particularly when a high IOU threshold is used for accurate bounding-box detection required in real-world applications. Faster R-CNN with our proposed SF-PSPyramid also outperforms most existing methods on the DeepPCB dataset. By aggregating multi-scale features, the SF-PSPyramid model significantly improves the performance of Faster R-CNN, resulting in an increase of Average Precision at 85% IoU (AP85) by more than 10%. In addition, the non-referential SF-PSPyramid model achieves an Average Precision of 98.7 at 33% IoU, achieving a better AP than the DeepPCB Tang's referential method [12].

The above experiment results prove that the SF-PSPyramid model can effectively improve the performance of existing object detectors for PCB defect detection, even when template images are not available. The effectiveness of SF-PSPyramid and multi-scale training in aggregating multi-scale features and improving the localization performance of Faster R-CNN highlights its potential in the field of PCB defect detection. Our results on DeepPCB indicate that, first, SF-PSPyramid, specifically designed for tiny defects, outperforms all existing baseline models, because high semantic information is added to large feature maps to assist in the prediction. Second, our model achieves higher overall mAP and AP at high IOU thresholds, which indicates a better match between predicted and ground-truth defect boxes, as SF-PSPyramid helps to weight important regions and suppress irrelevant regions. Furthermore, pixel shuffle upsampling can better preserve information before fusing two feature maps.

4.3.2. Results of referential methods on the DeepPCB dataset

Unlike Tang's method [12], which adds template information after extracting features from VGG-tiny, our method concatenates the template image with the test input image to form the input. By doing so, it can preserve more information from both the template image and the test image. Table 2 tabulates the results based on referential methods using our proposed methods. As Tang's method used AP33, all results are also computed with AP33. Faster R-CNN with SF-PSPyramid achieves better performance than Tang's method in every category.

4.3.3. Results of all IoU on the TDD dataset

Table 3 presents a comparative analysis of the mean average precision (mAP) of the proposed SF-PSPyramid and several baseline methods at different IoU thresholds. Compared to the state-of-the-art model, a modified YOLOv5 [23], which achieves a very satisfactory AP50, Faster R-CNN with our SF-PSPyramid can reduce the error of AP at IoU 50 from 0.83% (100%–99.1%) to 0.7% (100%–99.3%). This represents a 16% reduction, in terms of the error in AP50.

Our results demonstrate that SF-PSPyramid significantly outperforms the previous models, in terms of AP@50:5:85. In particular, when the IOU is higher than 70, which represents a challenging scenario for the baseline models, SF-PSPyramid outperforms the previous best model, CornerNet, by approximately 3%. This highlights the effectiveness of our proposed method in handling difficult situations when tiny defects are required to be located accurately. PSPyramid helps aggregate multi-scale information without upsampling distortion, and SF attention

Table 2

Comparison of referential methods on the DeepPCB dataset.

Model	open	short	mouse bite	spur	copper	pinhole	AP33
Image Processing [39]	88.2	87.6	90.3	88.9	91.5	89.2	89.3
YOLO [40]	90.5	92.0	93.1	93.3	94.9	92.6	92.6
SSD [36]	93.1	94.5	95.7	96.7	96.9	98.7	95.9
CornerNet [38]	98.5	95.8	96.7	97.8	98.8	99.3	97.8
GPP-MP [12]	98.5	98.5	99.1	98.2	98.5	99.4	98.6
Faster R-CNN [14]	96.8	95.4	97.9	98.7	97.4	99.5	97.6
Faster R-CNN with SF-PSPyramid	99.8	99.5	99.7	99.5	99.5	100.0	99.7

dynamically weights the information for better prediction of tiny defects.

4.3.4. Performance of different methods under IoU 50 on the TDD dataset

Table 4 shows the performance of different methods and our proposed Faster-RCNN with SF-PSPyramid on the TDD dataset. In general, our method achieves the best performance in almost all categories, compared to the second-best method, TDDNet [18]. Although our model uses a smaller ResNet-50 as the backbone, it still outperforms TDDNet, which employs the larger ResNet-101. All results are based on IoU at 0.5 because the original TDDNet only has results for IoU at 0.5. Since the TDD dataset is large, more epochs are required to train deep models. All methods in this experiment are trained for a certain number of epochs such that they achieve the best performance.

Compared to the DeepPCB dataset, the PCB images in the TDD dataset are colour images, providing computer vision models with additional information to identify defects. However, the defects in the TDD dataset are tiny, so the detection needs to be more precise. Our SF-PSPyramid network provides a solution to this challenge, achieving more than 1% improvement, in terms of AP@50:5:85 over Faster R-CNN. The ability of SF-PSPyramid to accurately detect tiny defects makes it an excellent tool for addressing the specific requirements of the TDD dataset. SF-PSPyramid can effectively inject semantic information into feature maps at shallow layers. This certainly contributes to the improvement. Furthermore, pixel shuffle upsampling can preserve and convey semantic information during upsampling.

4.4. Ablation study

In this study, we conducted an ablation analysis to determine the effect of our proposed components on the performance of a PCB defect detection model. We validate the design of our SF-PSPyramid model by separately incorporating novel components with different loss functions. To improve performance, we also explore the use of different fusion schemes. Furthermore, we evaluate our method using object detection techniques, such as focal loss, hard negative mining, and k -means clustering anchors. We have also used different model neck designs and model pruning methods to form lightweight models.

Table 1

Comparison of different non-referential methods on the DeepPCB dataset.

Model	AP50	AP55	AP60	AP65	AP70	AP75	AP80	AP85	AP@50:5:85
SSD [36]	97.4	96.5	95.2	92.6	88.5	78.7	60.8	37.5	80.9
CornerNet [38]	89.6	89.2	88.9	88.0	86.1	82.1	73.1	56.5	81.7
RetinaNet [37]	96.9	96.7	96.3	95.1	92.5	86.4	74.4	53.6	86.5
FCCN GWP [13]	96.9	96.8	96.5	95.7	94.2	89.6	79.5	58.8	88.5
Faster R-CNN [14]	97.0	96.6	96.0	95.2	94.0	90.0	80.5	61.6	88.8
Faster R-CNN with SF-PSPyramid	98.6	98.5	98.3	98.0	96.9	94.6	88.1	72.7	93.2

Table 3

Comparison of different methods on the TDD PCB dataset.

Model	AP50	AP55	AP60	AP65	AP70	AP75	AP80	AP85	AP@50:5:85
SSD [36]	98.6	98.1	96.0	89.6	76.9	52.7	24.4	6.0	67.8
CornerNet [38]	98.2	97.9	96.1	91.1	79.8	59.4	33.2	10.6	70.8
RetinaNet [37]	94.7	91.9	86.9	75.7	57.8	34.8	14.1	3.1	57.4
Coordinate Feature Refinement [22]	97.9	NA							
Faster R-CNN [14]	98.1	97.1	94.4	86.9	72.9	51.5	26.0	8.7	67.0
TDDNet [18]	98.9	NA							
YOLOv5 + new FPN + modified CIOU Loss [23]	99.2	NA							
Faster R-CNN with SF-PSPyramid	99.3	98.9	97.8	93.9	84.6	64.2	36.9	11.8	73.4

Table 4

Comparison of different methods for IOU 50 on the TDD PCB dataset, with the best results highlighted in bold.

Model	Open circuit	Short	Mouse bite	Spurious copper	Spur	Missing hole	AP50
SSD [36]	99.5	96.9	99.4	98.4	98.6	99.1	98.6
CornerNet [38]	99.2	97.4	98.3	97.9	97.6	99.0	98.2
RetinaNet [37]	93.5	95.4	94.0	93.1	94.6	97.8	94.7
Faster R-CNN [14]	98.5	96.4	98.3	98.3	97.5	99.3	98.1
TDDNet [18]	98.6	98.5	99.2	98.7	99.0	99.4	98.9
Faster R-CNN with SF-PSPyramid	99.4	98.3	99.6	99.4	99.6	99.4	99.3

4.4.1. Results of adding different novel components

The results shown in Table 5 highlight the improvements obtained by incorporating PSPyramid and SF attention into the baseline Faster R-CNN with ROI alignment. We focus on these results because the data show that SF-PSPyramid significantly outperforms the baseline on AP80 and AP85, which are the most difficult cases. The results show that PSPyramid has a high impact on performance, while SF attention achieves a smaller improvement. This demonstrates the effectiveness of PSPyramid as a multi-scale feature fusion module and its ability to perform well in localization tasks at high IoU thresholds. SF attention can further improve the AP at high IoU because it mainly fuses the feature maps at shallow layers to make the predicted bounding boxes more accurate. The design choices are crucial for the accurate detection of tiny defects on both the DeepPCB and TDD datasets.

4.4.2. Results of fusing feature maps using different CNN attention mechanisms in the proposed SF attention

Table 6 shows the performance of using and not using our proposed SF attention, and also compares SF attention with other state-of-the-art attention mechanisms for fusing two feature maps in the model neck. For Coordinate Attention [35], SE [25], and CBAM [27], the attention mechanism is applied to each feature map before summation and can improve performance significantly. However, these traditional attention mechanisms are not designed for fusing feature maps. Our proposed SF attention is effective in fusing two feature maps, as it learns optimal weights to combine feature maps, while other methods are not designed for combining two feature maps.

4.4.3. Results of using different loss functions

Various loss functions can be used to train object detection models for predicting bounding boxes around objects in images. Mean Squared Error (MSE) and Mean Absolute Error (MAE) are common choices for regression tasks, including bounding-box prediction. These loss functions compare the predicted bounding-box parameters (e.g., the coordinates, width, and height) with those of the ground-truth bounding

box and penalize the model for errors.

Another loss function that can be used for bounding-box prediction is the Intersection-over-Union (IoU) loss. This loss measures the overlap between the predicted bounding box and the ground-truth box and is calculated as the ratio of the intersection of the two boxes to their union. The IoU loss takes into account the integrity of the object itself and is commonly used in object detection tasks. The Generalized IoU (GIoU) loss builds upon the IoU loss by also considering the shape and orientation of predicted bounding boxes. This is particularly useful in cases where the predicted bounding box may be rotated or skewed relative to the ground-truth box.

In object detection, various loss functions can be used to improve the accuracy of bounding-box predictions. The appropriate loss function is dependent on the task requirements and desired model behaviour. In this study, we utilize L1-loss as the regression loss. The results in Table 7 demonstrate that L1-Loss performs best in PCB defect detection when the proposed SF-PSPyramid is used. In Table 8, the results on the TDD dataset are also consistent with the DeepPCB dataset. It shows that using L1-loss with the SF-PSPyramid works better.

In object detection, L1-loss is frequently used as a regression loss function due to its robustness in the presence of outliers. This makes L1-loss a more suitable choice for tasks with a large number of outliers, as components may also look like defects. L1-loss is also better than IOU loss and GIoU loss because the geometry, and aspect ratio are always similar in PCB images.

4.4.4. Results of other object detection techniques

Table 9 shows the results of using different techniques to boost defect detection performance. In Yolo [42] and TDDNet [18], *k*-means clustering is used to determine five priors for anchors. However, specific anchors cannot help improve performance in our experiments, because most defects are similar in aspect ratio. Focal loss and OHEM, which guide the model to pay more attention to hard examples, are also evaluated. However, the performance of OHEM and focal loss depend on the distribution of the data. And the DeepPCB dataset has a balanced class distribution and therefore OHEM and Focal loss may not be necessary. The result shows that no performance gain is achieved by adding the aforementioned methods.

4.4.5. Results of modifying the model architecture

Table 10 shows the impact of the P6 neck on the proposed model. The P6 layer, which is derived from the P5 layer, is utilized to detect the largest PCB defects. Removing the P6 layer leads to a decline in

Table 5

Ablation study of using our novel PSPyramid and SF-PSPyramid.

Model	AP80	Improvement	AP85	Improvement
Faster R-CNN [14]	80.5	–	61.6	–
+PSPyramid	87.4	8.6%	72.7	18.0%
+SF-PSPyramid	88.1	9.4%	72.7	18.0%

Table 6

Ablation study of using different attention mechanisms and the proposed SF attention on the TDD dataset.

Attention mechanism	AP50	AP55	AP60	AP65	AP70	AP75	AP80	AP85	AP@50:5:85
Without attention	98.1	97.1	94.4	86.9	72.9	51.5	26.0	8.7	67.0
Coordinate Attention [41]	98.9	98.2	96.4	90.2	77.6	56.3	27.7	7.4	69.1
SE [31]	99.1	98.2	96.6	90.9	79.3	57.3	29.4	8.2	69.8
CBAM [33]	99.3	98.7	97.2	93.2	83.0	62.5	35.6	11.2	72.6
SF attention	99.3	98.9	97.8	93.9	84.6	64.2	36.9	11.8	73.4

Table 7

Comparison of different regression losses on the DeepPCB dataset.

Loss function	AP50	AP55	AP60	AP65	AP70	AP75	AP80	AP85	AP@50:5:85
Smooth L1	96.5	96.3	96.1	95.5	94.4	91.5	84.1	68.8	90.4
IoU Loss	97.7	97.6	97.1	96.7	95.6	93.1	86.4	69.8	91.8
GloU Loss	98.2	98.1	97.8	97.2	96.2	93.3	86.9	70.9	92.3
L1-Loss	98.6	98.5	98.3	98.0	96.9	94.6	88.1	72.7	93.2

Table 8

Comparison of different regression losses on the TDD dataset.

Loss function	AP50	AP55	AP60	AP65	AP70	AP75	AP80	AP85	AP@50:5:85
Smooth L1	98.9	98.3	96.2	91.2	77.9	54.6	27.3	7.5	69.0
IoU Loss	99.1	98.6	97.1	92.2	80.3	57.7	30.0	8.1	70.4
GloU Loss	99.2	98.4	96.8	91.3	79.3	56.1	28.6	7.2	69.6
L1-Loss	99.3	98.9	97.8	93.9	84.6	64.2	36.9	11.8	73.4

Table 9Comparison of using k -means clustering anchor, focal loss, OHEM on the DeepPCB dataset.

Model	AP50	AP55	AP60	AP65	AP70	AP75	AP80	AP85	AP@50:5:85
Ours	98.6	98.5	98.3	98.0	96.9	94.6	88.1	72.7	93.2
+ k -means	98.0	97.7	97.3	96.9	95.8	93.4	86.2	70.3	92.0
+Focal loss	95.8	95.7	95.4	94.7	93.4	90.6	83.6	68.2	89.7
+OHEM	97.9	97.6	97.3	96.9	95.9	93.2	85.9	70.0	91.8

Table 10

Ablation study removing the P6 layer and using the de-aliasing convolution.

Model	AP50	AP55	AP60	AP65	AP70	AP75	AP80	AP85	AP@50:5:85
Removing the P6 layer	97.9	97.8	97.4	97.1	95.9	93.3	85.4	70.9	92.0
Dealiasing	97.7	97.6	97.2	96.8	95.8	93.5	85.8	70.5	91.9
Ours	98.6	98.5	98.3	98.0	96.9	94.6	88.1	72.7	93.2

performance by 1.2 AP@50:5:85 because the neck is reduced to four layers only. The P6 feature map still contributes to the model's ability to identify the location of defects, even if no large defects are present in a PCB image. Furthermore, some feature pyramid networks, such as [27], may employ an additional 3×3 convolutional layer to reduce aliasing effects in fused features. Our method includes 3×3 convolutional layers after each fused feature, but this additional layer does not improve performance. The reason for this is that our PSPyramid already uses convolution when increasing the number of channels for pixel shuffling, effectively performing de-aliasing within the convolutional kernel, thus eliminating the need for an extra kernel. This finding highlights the

effectiveness of our SF-PSPyramid for feature extraction of tiny defects.

4.4.6. Results of pruned models

The feasibility of using a lightweight model to achieve performance similar to the full models is studied by performing model pruning on the Resnet50 backbone. Table 11 shows that even with only 20% of the weights retained, the pruned model still outperforms all of the other non-referential-based full models. The pruned model outperforms the baseline models when the IoU is below 70, accentuating the significance of the model neck in retaining the performance for defect detection. These findings also highlight the advantage of SF-PSPyramid for overall

Table 11

Results of the pruned models.

Model	AP50	AP55	AP60	AP65	AP70	AP75	AP80	AP85	AP@50:5:85
SSD [36]	97.4	96.5	95.2	92.6	88.5	78.7	60.8	37.5	80.9
CornerNet [38]	89.6	89.2	88.9	88.0	86.1	82.1	73.1	56.5	81.7
RetinaNet [37]	96.9	96.7	96.3	95.1	92.5	86.4	74.4	53.6	86.5
FCCN GWP [12]	96.9	96.8	96.5	95.7	94.2	89.6	79.5	58.8	88.5
Faster R-CNN [14]	97.0	96.6	96.0	95.2	94.0	90.0	80.5	61.6	88.8
Pruned SF-PSPyramid	97.7	97.5	97.1	96.5	94.2	89.3	79.3	61.1	89.1

AP improvement and the difficulty of defect detection at high IOU thresholds.

4.5. Model sizes and computational complexities

Table 12 tabulates the number of parameters and the computational requirement, in terms of GFLOPs, for five different models. The data reveals that SF-PSPyramid, despite having more parameters, still manages to achieve a GFLOP value that remains competitive, particularly when considering the balance between parameter efficiency and model accuracy. In comparison to SSD, SF-PSPyramid has almost three times more parameters but reduces GFLOPs by 10%, highlighting a more efficient use of computing resources.

Upon further examination, it is revealed that, despite being rooted in the Faster R-CNN framework, the proposed model requires nearly six times fewer GFLOPs. This signifies a substantial enhancement in computational efficiency without compromising performance, as evidenced by its AP50 score on the DeepPCB dataset. This efficiency translates into a reduction in inference times, making it more suitable for real-time applications in industrial settings. Although RetinaNet has only half the parameters of SF-PSPyramid, the GFLOPs of RetinaNet are not proportionally reduced, which can be attributed to the inherent computational optimization of SF-PSPyramid.

Considering the importance of defect detection in PCB manufacturing, the increased number of parameters in SF-PSPyramid is justified, particularly in light of its superior accuracy in defect detection. The model's scalability and adaptability to various PCB layouts, as well as its efficient resource utilization, suggest its readiness for deployment in contemporary industrial environments, where speed and accuracy are imperative.

4.6. Integration with manufacturing workflows

To facilitate this process in a production environment, a high-resolution camera is necessary to capture detailed PCB images, and an inference server must be installed at the end of the production line to process these images in real-time. Upon collecting and labelling a sufficient number of PCB images, the proposed algorithm can initially be trained using a standard consumer-grade GPU or via cloud-based services.

Even with its increased number of parameters, the computational requirements of the model remain manageable. It is recommended to use the model in conjunction with a low-end Nvidia GPU, such as an RTX 4060, with more than 8 GB of VRAM, to allow the entire model to fit on the GPU and achieve real-time inference capabilities. Subsequently, the trained model is deployed on an inference server, which can be a GPU or a cloud-based server, for real-time defect detection during the PCB manufacturing process.

To accommodate multiple production lines, the system's capabilities can be expanded by adding inference servers or leveraging more cloud computing services to handle a larger volume of images. Moreover, in the case of a high-speed production line, the system can be vertically enhanced; this is achieved by upgrading to more powerful hardware or purchasing more powerful cloud computing services to speed up the

inference process. This adaptable setup, which capitalizes on widely accessible technologies, ensures cost efficiency, thereby enhancing the model's applicability for widespread use in automated PCB inspection systems.

4.7. Limitations

While our model can be deployed in industrial contexts, it is not without limitations. As with other models, introducing a new PCB demands a robust dataset of images and annotations to train a fresh model. The demonstrated generalizability of the SF-PSPyramid model on the DeepPCB and TDD datasets suggests promising performance on unseen datasets, assuming a similar volume of training data is available. Despite the possibility of leveraging transfer learning from existing PCB models, detection performance may vary with the disparity of PCB layouts. If the production timeframe is short and defect detection is imperative, additional manpower for annotation may be needed. Thus, it may not be suitable for small-scale manufacturing.

Installation of appropriate hardware on the production line may require modifications to the current setup, including the integration of cameras and an inference server. Additionally, ambient lighting in the production environment may impact the efficacy of the defect detector, particularly for reflective solder surfaces, necessitating the creation of a controlled environment for optimal functionality. In our experiments, we observed that our model's performance decreases as the IOU threshold increases. This indicates an opportunity for further refinement to improve the model's precision at higher IOU thresholds.

This study focuses on utilizing SF-PSPyramid alone for PCB inspection; however, its application potential extends to other facets of manufacturing that demand high-precision detection of tiny objects, such as verifying component placement, inspecting wire bonds, and inspecting solder paste applications. Conducting further experiments in these different areas can significantly bolster the standards within the electronics manufacturing industry.

4.8. Visualizations

4.8.1. Visualization of bounding boxes

When evaluating the performance of our method for PCB defect detection, we compared it with Faster R-CNN using the DeepPCB dataset. The motivation behind this comparison stems from the fact that our model borrows elements from Faster R-CNN, both have well-designed model architectures.

The results of this comparison are shown in Fig. 7, which depicts that our model achieves superior performance compared to the baseline models. In the first row, only our proposed method can successfully detect the spur in the bottom right corner. This is attributed to the use of pixel shuffle upsampling, which provides a larger receptive field and enables more effective differentiation between foreground and background.

The Faster R-CNN model, in the second row, struggles to classify the background into the "open" class. Furthermore, the Faster R-CNN model generates the false defect "short" in the background. This indicates that our model can not only detect defects on the largest feature map but also use larger receptive fields on smaller feature maps to help decide whether neighbouring areas are part of the circuit. In the third row, Faster-RCNN misclassifies two background circles as the class "copper". The misclassified circles are slightly smaller than other circles in the background. This shows that the proposed SF-PSPyramid can effectively aggregate spatial and scale information during training.

4.8.2. Visualization of attention maps

In this section, we compare the visual attention maps of Faster-RCNN, PSPyramid, and SF-PSPyramid on the DeepPCB dataset. We used Eigen-CAM [43] as the visualization method. The colour intensity is determined by a linear combination of the weights of the selected

Table 12

The number of parameters and floating-point operations per second (FLOPs) of different models.

Model	AP50 (DeepPCB)	Parameters (M)	GFLOPs
SSD [36]	97.4	25.27	138.38
CornerNet [38]	89.6	200.96	705.82
RetinaNet [37]	96.9	36.23	82.75
Faster R-CNN [14]	97.0	33.57	775.61
Faster R-CNN with SF-PSPyramid	98.6	72.06	124.78

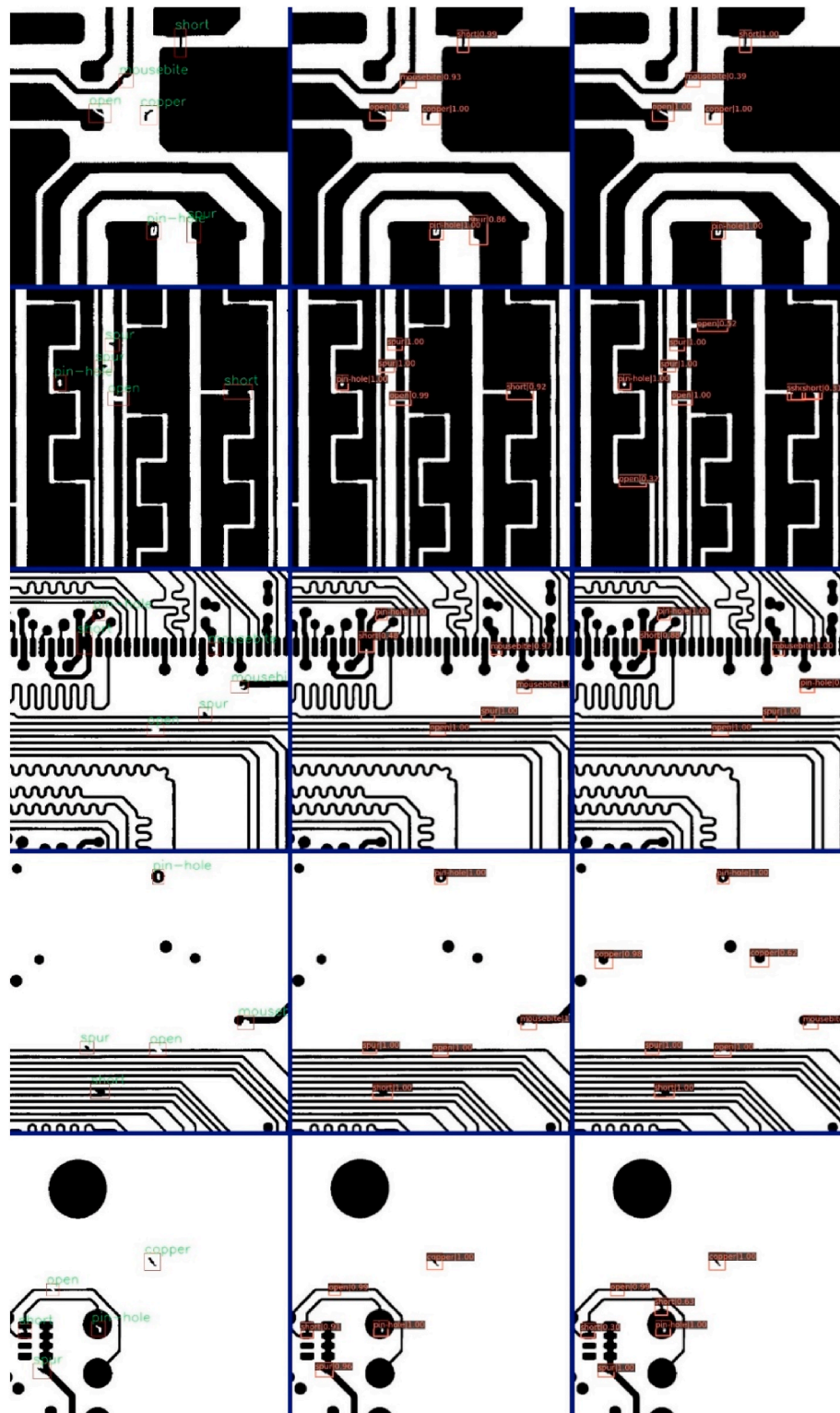


Fig. 7. Comparison of visual results for different models. First column: ground truths. Second column: by the proposed Faster R-CNN with SF-PSPyramid. Third column: by Faster R-CNN.

principal components from principal component analysis (PCA). The intensity of the colour represents the contribution of each eigenvector to the overall image. By using PCA, Eigen-CAM can identify the most important features that contribute to the final decision of the CNN model. In the first row of Fig. 8, we can see that Faster-RCNN does not perform well on the tiny defects in the DeepPCB dataset. Most of the attention is not focused on the tiny defects, and it misidentifies the

printed text on the left. Both PSPyramid and SF-PSPyramid perform well on tiny defects because of the strong attention around the defects. However, with SF attention, the large defects on the right are also strongly focused in the attention map. This means that PSPyramid may have difficulty learning in small feature maps, while SF attention can help alleviate this problem by reweighting the importance. In the second row of Fig. 8, Faster RCNN misclassifies the printed text and does not pay

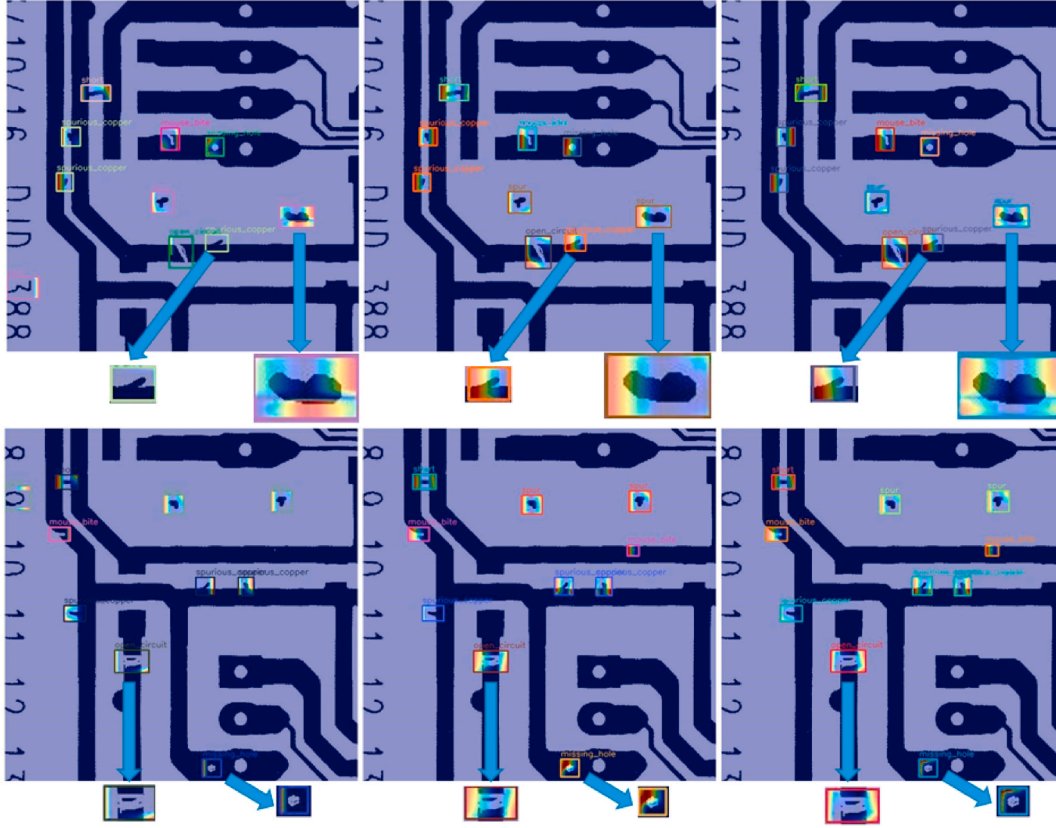


Fig. 8. Eigen-CAM visualization results on DeepPCB. Left: Faster-RCNN, Middle: PSPyramid, and Right: SF-PSPyramid.

strong attention to the tiny defects. Both PSPyramid and SF-PSPyramid have stronger attention around the defects. Similarly, SF-PSPyramid also generates a better attention map on spurious copper.

Fig. 9 shows the detection results on the TDD dataset. We can see that faster RCNN can only detect part of the defects. However, as illustrated in the attention maps, both of our proposed models can generate stronger and more complete attention around the defects.

5. Conclusion

In this paper, we propose an object detection model, based on Faster R-CNN, for accurate PCB defect detection. To address the challenge of detecting tiny defects and degraded localization with a high IoU on PCBs, we introduce a novel module, called SF-PSPyramid. SF-PSPyramid and multi-scale training efficiently aggregate input and feature maps of

multiple sizes, making the model scale invariant to the feature maps across different layers of the backbone. Additionally, SF attention helps to merge and select important feature maps for tiny PCB defect detection.

We conducted experiments on the DeepPCB and TDD PCB datasets and found that using Soft-NMS and multi-scale training can significantly improve the model performance. Our proposed SF-PSPyramid outperforms state-of-the-art methods on both datasets, even with a ResNet-50 backbone rather than ResNet-101. The pruned lightweight model can also outperform most of the baseline models. Specifically, on the DeepPCB dataset, our non-referential method can achieve similar performance to template-based methods, making it especially useful in production environments, where template images are not available. On the TDD PCB dataset, our model outperforms state-of-the-art methods, achieving at least a 16% reduction in mAP error, compared to the state-

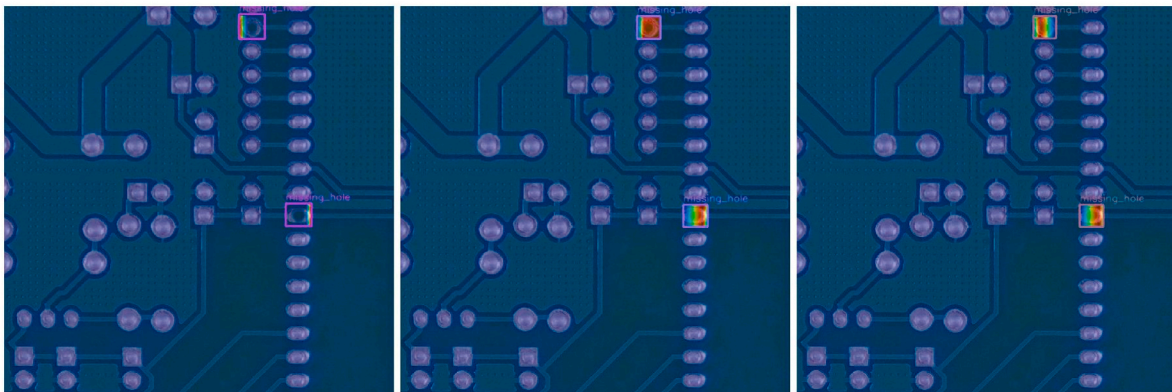


Fig. 9. Eigencam Attention visualization result on TDD. Left: Faster-RCNN, Middle: PSPyramid, and Right: SF-PSPyramid.

of-the-art methods.

CRedit authorship contribution statement

Ka Chun Fung: Conceptualization, Writing – original draft. **Kai-Wen Xue:** Conceptualization, Methodology. **Cheung-Ming Lai:** Investigation, Project administration. **Kwan-Ho Lin:** Investigation, Project administration, Writing – review & editing. **Kin-Man Lam:** Supervision, Validation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

References

- [1] H.Q.T. Ngo, Design of automated system for online inspection using the convolutional neural network (CNN) technique in the image processing approach, *Results in Engineering* (2023) 101346.
- [2] T. Sahar, et al., Anomaly detection in laser powder bed fusion using machine learning: a review, *Results in Engineering* (2022) 100803.
- [3] D. Tabernik, S. Šela, J. Skvarč, D. Škočaj, Segmentation-based deep-learning approach for surface-defect detection, *J. Intell. Manuf.* 31 (3) (2020) 759–776.
- [4] X. Li, C. Duan, Y. Zhi, P. Yin, Wafer crack detection based on yolov4 target detection method, in: *Journal of Physics: Conference Series*, vol. 1802, IOP Publishing, 2021 022101, 2.
- [5] A. Bochkovskiy, C.-Y. Wang, H.-Y.M. Liao, Yolov4: optimal speed and accuracy of object detection, *arXiv preprint arXiv:2004.10934* (2020).
- [6] J. Ye, Two-stage Soldering Defect Detection with Deep Learning, 2019.
- [7] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Commun. ACM* 60 (6) (2017) 84–90.
- [8] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, *arXiv preprint arXiv:1409.1556* (2014).
- [9] C. Szegedy, et al., Going deeper with convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 1–9.
- [10] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [11] X. Chen, Y. Wu, X. He, W. Ming, A Comprehensive Review of Deep Learning-Based PCB Defect Detection, *IEEE Access*, 2023.
- [12] S. Tang, F. He, X. Huang, J. Yang, Online PCB Defect Detector on a New PCB Defect Dataset, 2019 *arXiv preprint arXiv:1902.06197*.
- [13] Y. Gao, J. Lin, J. Xie, Z. Ning, A real-time defect detection method for digital signal processing of industrial inspection applications, *IEEE Trans. Ind. Inf.* 17 (5) (2020) 3450–3459.
- [14] S. Ren, K. He, R. Girshick, J. Sun, Faster r-cnn: towards real-time object detection with region proposal networks, *Adv. Neural Inf. Process. Syst.* 28 (2015).
- [15] S. Khalilian, Y. Hallaj, A. Balouchestani, H. Karshenas, A. Mohammadi, Pcb defect detection using denoising convolutional autoencoders, in: 2020 International Conference on Machine Vision and Image Processing (MVIP), IEEE, 2020, pp. 1–5.
- [16] Z. Zeng, B. Liu, J. Fu, H. Chao, Reference-based defect detection network, *IEEE Trans. Image Process.* 30 (2021) 6637–6647.
- [17] H. Xie, Y. Kuang, X. Zhang, A high speed AOI algorithm for chip component based on image difference, in: 2009 International Conference on Information and Automation, IEEE, 2009, pp. 969–974.
- [18] R. Ding, L. Dai, G. Li, H. Liu, TDD-net: a tiny defect detection network for printed circuit boards, *CAAI Transactions on Intelligence Technology* 4 (2) (2019) 110–116.
- [19] A. Shrivastava, A. Gupta, R. Girshick, Training region-based object detectors with online hard example mining, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 761–769.
- [20] P. Malge, R. Nadaf, PCB defect detection, classification and localization using mathematical morphology and image processing tools, *International journal of computer applications* 87 (9) (2014).
- [21] J. Zhou, et al., Toward TR-PCB bubble detection via an efficient attention segmentation network and dynamic threshold, *IEEE Trans. Instrum. Meas.* 72 (2023) 1–12.
- [22] J. Yang, Z. Liu, W. Du, S. Zhang, A PCB defect detector based on coordinate feature refinement, *IEEE Trans. Instrum. Meas.* 72 (2023) 1–10.
- [23] J. Lim, J. Lim, V.M. Baskaran, X. Wang, A deep context learning based PCB defect detection model with anomalous trend alarming system, *Results in Engineering* 17 (2023) 100968.
- [24] X. Liyun, L. Boyu, M. Hong, L. Xingzhong, Improved faster R-CNN algorithm for defect detection in powertrain assembly line, *Procedia CIRP* 93 (2020) 479–484.
- [25] B. Xia, H. Luo, S. Shi, Improved faster R-CNN based surface defect detection algorithm for plates, *Comput. Intell. Neurosci.* 2022 (2022).
- [26] K. He, X. Zhang, S. Ren, J. Sun, Spatial pyramid pooling in deep convolutional networks for visual recognition, *IEEE Trans. Pattern Anal. Mach. Intell.* 37 (9) (2015) 1904–1916.
- [27] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie, Feature pyramid networks for object detection, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 2117–2125.
- [28] G. Ghiasi, T.-Y. Lin, Q.V. Le, Nas-fpn: learning scalable feature pyramid architecture for object detection, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 7036–7045.
- [29] S. Liu, L. Qi, H. Qin, J. Shi, J. Jia, Path aggregation network for instance segmentation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8759–8768.
- [30] W. Shi, et al., Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 1874–1883.
- [31] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7132–7141.
- [32] X. Li, W. Wang, X. Hu, J. Yang, Selective kernel networks, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 510–519.
- [33] S. Woo, J. Park, J.-Y. Lee, I.S. Kweon, Cbam: convolutional block attention module, in: *Proceedings of the European Conference on Computer Vision, ECCV*, 2018, pp. 3–19.
- [34] K. He, G. Gkioxari, P. Dollár, R. Girshick, Mask r-cnn, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2961–2969.
- [35] N. Bodla, B. Singh, R. Chellappa, L.S. Davis, Soft-NMS—improving object detection with one line of code, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 5561–5569.
- [36] W. Liu, et al., Ssd: single shot multibox detector, in: *European Conference on Computer Vision*, Springer, 2016, pp. 21–37.
- [37] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.
- [38] H. Law, J. Deng, Cornernet: detecting objects as paired keypoints, in: *Proceedings of the European Conference on Computer Vision, ECCV*, 2018, pp. 734–750.
- [39] S.I. Putera, Z. Ibrahim, Printed circuit board defect detection using mathematical morphology and MATLAB image processing tools, in: 2010 2nd International Conference on Education Technology and Computer, vol. 5, IEEE, 2010. V5-359-V5-363.
- [40] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: unified, real-time object detection, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.
- [41] Q. Hou, D. Zhou, J. Feng, Coordinate attention for efficient mobile network design, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 13713–13722.
- [42] J. Redmon, A. Farhadi, Yolov3: an incremental improvement, *arXiv preprint arXiv:1804.02767* (2018).
- [43] M.B. Muhammad, M. Yeasin, Eigen-cam: class activation map using principal components, in: 2020 International Joint Conference on Neural Networks (IJCNN), IEEE, 2020, pp. 1–7.

Update

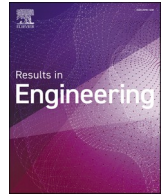
Results in Engineering

Volume 22, Issue , June 2024, Page

DOI: <https://doi.org/10.1016/j.rineng.2024.102045>

Contents lists available at [ScienceDirect](https://www.sciencedirect.com)

Results in Engineering

journal homepage: www.sciencedirect.com/journal/results-in-engineering

Corrigendum to “Improving PCB defect detection using selective feature attention and pixel shuffle pyramid” [Results. Eng. 21 (2024) 101992]

Ka Chun Fung^{a,b,*}, Kai-Wen Xue^b, Cheung-Ming Lai^b, Kwan-Ho Lin^b, Kin-Man Lam^{a,b}

^a The Hong Kong Polytechnic University (PolyU), Hong Kong

^b Centre for Advances in Reliability and Safety Limited (CAiRS), Hong Kong

The authors regret not including the following acknowledgement in the published version of the above-mentioned article

Acknowledgement

The work presented in this article is supported by Centre for

Advances in Reliability and Safety (CAiRS) admitted under AIR@-InnoHK Research Cluster.

The authors would like to apologise for any inconvenience caused.

DOI of original article: <https://doi.org/10.1016/j.rineng.2024.101992>.

* Corresponding author. The Hong Kong Polytechnic University (PolyU), Hong Kong.

E-mail address: ka-chun-ben.fung@connect.polyu.hk (K.C. Fung).

<https://doi.org/10.1016/j.rineng.2024.102045>

Available online 20 March 2024

2590-1230/© 2024 Published by Elsevier B.V.